

Documentação do Protótipo de Front-end

Projeto: EscolaAlerta

Dupla: Mateus Farias Bentes e Mariana Idalina Bezerra Vidinho

1. Introdução

Este documento descreve a implementação do front-end do protótipo EscolaAlerta: arquitetura de telas, motor de classificação de risco, modelo de dados, sistema de design e catálogo de componentes React. O protótipo cobre dois perfis de uso com jornadas independentes a partir de uma mesma base de dados em memória — orientador escolar e responsável — e existe em duas versões-fonte mantidas em paralelo, descritas na seção 2.

Perfil	O que faz	Telas
Orientador escolar	Cadastra alunos e registra indicadores (nota, falta, ocorrência, saída antecipada). Cada registro recalcula o risco do aluno em tempo real.	dashboard → cadastro / registro de indicador
Responsável	Acompanha o nível de risco do dependente, lê o plano de ação sugerido pela escola quando há risco e devolve feedback sobre a resolução.	painel de risco → plano de ação

Observação: a tela de login é decorativa — os dois botões apenas escolhem qual perfil simular. Não há autenticação real (ver seção 10).

Este protótipo também incorpora as recomendações objetivas do documento "Documentação de Front-end — IHC e UX/UI" (avaliação heurística de Nielsen e auditoria WCAG/e-MAG): explicação do cálculo de risco visível antes da tela de plano de ação (`ExplicacaoRisco`), motivo principal exposto já no painel do responsável, busca e filtro por risco na lista de alunos, edição e exclusão de indicadores já lançados com confirmação inline, validação explícita da faixa de nota (0–10) e associação de `aria-describedby` entre botões desabilitados e o texto que explica por que estão desabilitados. Com essas mudanças, o protótipo atende integralmente às recomendações do documento.

2. Stack e Arquivos

O protótipo existe em duas versões paralelas do mesmo front-end, mantidas manualmente em sincronia: mesma árvore de componentes, mesmo motor de regras, mesmos tokens visuais.

2.1 escolaalerta-prototipo.html

Arquivo único e autocontido: React 18, ReactDOM e Babel Standalone carregados via CDN (`cdnjs`), com o componente `App` escrito dentro de uma tag `<script type="text/babel">` e transpilado no próprio navegador. Ícones via Font Awesome 6. Não requer instalação nem servidor — basta abrir o arquivo.

2.2 escolaalerta-prototipo.jsx

Mesma aplicação como módulo ES (`import/export default`), pensada para entrar em um projeto Vite, Create React App ou Next.js já existente. Ícones via `lucide-react` em vez de classes de fonte. Os tokens de estilo vivem em um componente `<GlobalStyle>` que injeta a mesma folha de estilos do arquivo `.html`.

2.3 Por que dois arquivos

O `.html` serve como demonstração imediata (apresentação, avaliação, revisão de design) sem qualquer configuração. O `.jsx` é o ponto de partida caso o protótipo evolua para integrar com a API Node.js e o banco PostgreSQL descritos em "Arquitetura da Solução e Padrões de Projeto". Eles não compartilham uma fonte única — alterações precisam ser replicadas manualmente nos dois arquivos (ver seção 10).

3. Arquitetura de Telas

Não há um roteador: a navegação é uma máquina de estados simples guardada em `useState` no componente `App`. A variável `tela` guarda o nome da tela atual; cada transição é uma chamada a `setTela(...)` disparada por uma prop de callback.

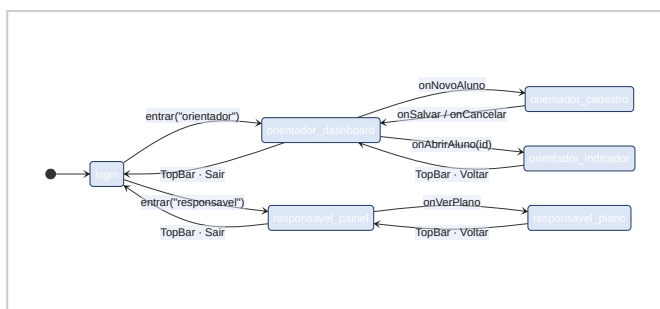


Figura 1: Diagrama de estados da navegação entre telas.

Valor de <code>tela</code>	Componente renderizado	Acessível via
login	<code><Login></code>	estado inicial
orientador-dashboard	<code><OrientadorDashboard></code>	login como orientador
orientador-cadastro	<code><CadastrarAluno></code>	botão "Cadastrar aluno"
orientador-indicador	<code><RegistrarIndicador></code>	clique em um aluno da lista
responsavel-painel	<code><PainelResponsavel></code>	login como responsável
responsavel-plano	<code><PlanoDeAcao></code>	botão "Ver plano de ação"

Observação: o fluxo do responsável está fixo em `ALUNO_DEMO_ID` ("a1", Rafael) — não existe seleção de dependente nem suporte a mais de um filho por responsável.

4. Motor de Classificação de Risco

A função `classificarRisco(aluno)` roda no cliente a cada renderização e a cada novo indicador registrado — não há chamada de API. Ela avalia três critérios de forma independente, atribui um peso a cada um e usa o maior peso para decidir o nível final e a causa principal.

Critério	Condição	Peso
Ocorrências	3 ou mais registradas	3
	1 ou 2 registradas	1
Faltas	5 ou mais	3
	3 ou 4	2
	1 ou 2	1
Notas abaixo de 6	2 ou mais disciplinas	3
	exatamente 1 disciplina	1

O **nível final** é derivado do maior peso entre os critérios que dispararam, e a **causa principal** é o critério que gerou esse peso (em empate, prevalece a ordem de avaliação: ocorrências → faltas → notas):

Peso máximo	Nível resultante	Efeito
≥ 3	Risco alto	Plano de ação acionado
$= 2$	Risco moderado	Plano de ação acionado
≤ 1 ou 0	Risco baixo	Sem plano de ação

4.1 Implementação

```
function classificarRisco(aluno) {
  // cada critério é avaliado de forma independente
  if (nOcorrencias >= 3) causas.push({ tipo: "ocorrencias", peso: 3 });
  else if (nOcorrencias >= 1) causas.push({ tipo: "ocorrencias", peso: 1 });

  if (nFaltas >= 5) causas.push({ tipo: "faltas", peso: 3 });
  else if (nFaltas >= 3) causas.push({ tipo: "faltas", peso: 2 });
  else if (nFaltas >= 1) causas.push({ tipo: "faltas", peso: 1 });

  if (nNotasBaixas >= 2) causas.push({ tipo: "notas", peso: 3 });
  else if (nNotasBaixas === 1) causas.push({ tipo: "notas", peso: 1 });

  // vence o critério de maior peso
  const maxPeso = Math.max(...causas.map(c => c.peso));
  const nivel = maxPeso >= 3 ? "alto" : maxPeso === 2 ? "moderado" : "baixo";
}
```

4.2 Efeito no plano de ação

Somente no momento de um *novo* registro (dentro do handler `onRegistrar` do componente `App`), se o nível recalculado deixa de ser "baixo", a aplicação grava `aluno.planoAcao` com a causa principal e as sugestões correspondentes, lidas de um dicionário estático `SUGESTOES`. Alunos já cadastrados em risco na carga inicial dos dados (mock) só recebem plano de ação após o próximo indicador ser registrado — o recálculo não é retroativo aos dados semeados.

5. Modelo de Dados

Todo o estado vive em `useState` no componente `App` — não há Context, Redux ou qualquer store externa. Um único array `alunos` é a fonte de verdade; atualizações passam por `atualizarAluno(id, updater)`, que mapeia o array trocando apenas o aluno afetado.

```
aluno = {
  id, nome, turma, responsavel,
  notas:      [{ id, disciplina, valor, periodo }],
  faltas:     [{ id, data, justificada, comprovante }],
  ocorrencias: [{ id, descricao, gravidade }],
  saidas:     [{ id, data, horario, motivo }],
  planoAcao:  { causa, sugestoes: [...], nivel } | null,
  feedbacks:  [{ id, status, observacao, data }],
}
```

Três alunos vêm pré-carregados por `dadosIniciais()` para demonstração: Rafael Costa Lima (já com 3 faltas e 1 ocorrência leve, portanto risco alto por faltas), Ana Beatriz Souza (sem indicadores negativos) e Diego Matos Pereira (uma falta justificada).

O campo `comprovante` em falta justificada foi adicionado a partir da pesquisa de validação com responsáveis ("Relatório de Resultados: Pesquisa de Validação"): um dos relatos livres descreveu não ter conseguido comprovar uma falta por não ter registrado a data e o motivo do atestado em nenhum lugar. Ao marcar "Falta justificada" no formulário, um campo de observação/comprovante aparece para registrar esse motivo no próprio sistema.

6. Sistema de Design

Paleta escura de superfícies neutras com um único acento de cor para ação (azul) e três cores semânticas reservadas exclusivamente para nível de risco — a cor nunca é usada como decoração, apenas para carregar significado. As três cores de risco foram calibradas para contraste mínimo de 4,5:1 (WCAG AA), validado par a par sobre seus fundos reais.

6.1 Cor

	Token	Valor		Token	Valor
	bg — fundo	#0B0D10		surface — cartão	#16181D
	accent — ação	#3D6BC7		text-muted	#9AA1AC

	Token	Valor		Token	Valor
	risco · baixo	#34D399		risco · moderado	#FBBF24
	risco · alto	#F87171		categoria · nota	#2F6193
	categoria · falta	#96681C		categoria · ocorrência	#C44536
	categoria · saída	#436E42			

Cada tipo de indicador (nota, falta, ocorrência, saída) usa sua cor de categoria em três pontos consistentes da interface — a aba correspondente, o marcador no histórico e a borda superior do card de formulário — reforçando a categoria sem depender de texto.

6.2 Tipografia

Papel	Fonte	Uso
Display / títulos	Space Grotesk	Cabeçalhos de tela, marca
Corpo / interface	IBM Plex Sans	Texto de UI, rótulos, corpo
Eyebrow / nota	Kalam	Legendas manuscritas de contexto
Dados / código	IBM Plex Mono	Notas, datas, trechos de código

6.3 Elevação e forma

Cartões usam sombra suave em duas camadas — nunca deslocamento duro — e cantos moderados: 12 px em cartões e linhas de lista, 8 px em botões, campos e abas, 6 px em badges. Nenhum elemento usa raio total (`border-radius: 999px`), decisão deliberada para não competir com os chips de risco.

6.4 Temas claro e escuro

Todo token de cor (fundo, superfícies, texto, acento, sucesso/aviso/perigo e as três cores de risco) é uma variável CSS (`--bg`, `--surface`, `--accent`, `--risco-alto` etc.) definida duas vezes: uma vez em `:root` (tema escuro, padrão) e outra em um seletor `[data-theme="light"]` aplicado ao elemento raiz do app quando o tema claro está ativo. Como os componentes só referenciam essas variáveis — nunca um valor hexadecimal direto —, qualquer componente novo herda os dois temas automaticamente, sem lógica condicional própria.

Um botão de alternância (ícone de sol/lua) fica disponível no `TopBar` em todas as telas pós-login e no canto superior direito da tela de login. O estado do tema vive em `useState` no componente `App` e não é persistido — cada carregamento do protótipo inicia no tema escuro. Os valores do tema claro reaproveitam a paleta navy/creme validada com contraste AA na primeira fase de acessibilidade deste protótipo (accent #223A5E, fundo #F6F3EC, risco baixo/moderado/alto em #2F6B3D/#8F5E0C/#A8382B).

7. Catálogo de Componentes

Todos os componentes são funções sem estado externo além do `useState` local — sem hooks customizados, sem memoização, sem Context. A comunicação entre telas é feita exclusivamente por callbacks recebidos via props a partir do componente `App`.

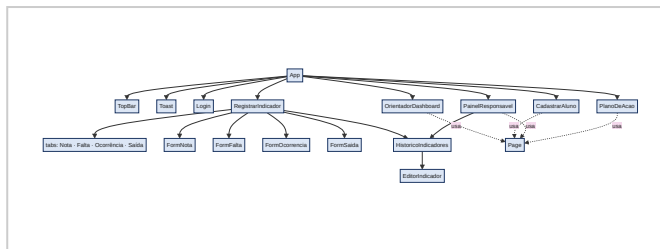


Figura 2: Árvore de composição dos componentes React.

Componente	Props	Responsabilidade
TopBar	perfil, onVoltar, onSair, tema, onAlternarTema	Cabeçalho fixo; mostra "Voltar" só quando onVoltar não é null; inclui o botão de alternância de tema.
Toast	texto, onFechar	Confirmação temporizada (3,2 s); role="status" + aria-live="polite".
Page	children, max	Wrapper <main> com largura máxima centralizada; alvo do skip-link.
Login	onEntrar, tema, onAlternarTema	Seleciona o perfil de simulação; botão de alternância de tema no canto superior direito.
OrientadorDashboard	alunos, onNovoAluno, onAbrirAluno	Lista de alunos como com busca por nome e filtro por nível de risco; cada linha com o badge de risco atual.
CadastrarAluno	onCancelar, onSalvar	Formulário controlado; onSalvar só habilita com nome e turma preenchidos.
RegistrarIndicador	aluno, onRegistrar, onEditar, onExcluir	Abas por tipo de indicador + histórico do aluno, com opção de editar ou excluir um registro já lançado.
FormNota / FormFalta / FormOcorrencia / FormSaida	onSalvar	Um <form> por tipo, com validação local e dica inline (associada via aria-describedby) quando desabilitado.
EditorIndicador	tipo, dados, onSalvar, onCancelar	Formulário de edição inline, um layout por tipo de indicador, pré-preenchido com os valores atuais; substitui a linha do histórico enquanto ativo.

Componente	Props	Responsabilidade
HistoricoIndicadores	aluno, onEditar, onExcluir (ambos opcionais)	Funde os quatro arrays de indicadores em uma linha do tempo única; exibe os ícones de editar/excluir apenas quando as respectivas props são passadas (só na visão do orientador — o painel do responsável permanece somente leitura).
ExplicacaoRisco	—	Disclosure nativo (<details>) que explica em linguagem simples como o motor de regras calcula o nível de risco; usado no painel do responsável e no registro de indicador.
PainelResponsavel	aluno, onVerPlano	Card de risco (com motivo principal já visível) + histórico; CTA para o plano só aparece se risco ≠ baixo.
PlanoDeAcao	aluno, onFeedback	Sugestões pedagógicas + formulário de feedback de resolução.

8. Acessibilidade

Itens verificados manualmente com cálculo de contraste WCAG por par cor-sobre-cor, não apenas por inspeção visual.

- **Rótulos de formulário:** todo `<label>` tem `htmlFor` ligado a um `id` real no controle — nenhum campo depende apenas de placeholder.
- **Envio por teclado:** todos os formulários de registro são `<form>` reais com `onSubmit`; a tecla Enter em qualquer campo envia o formulário.
- **Estado desabilitado explicado:** quando o botão de salvar está desabilitado, um texto abaixo dele informa exatamente o que falta preencher.
- **Landmarks semânticos:** `<header>` fixo, `<main id="main-content">` por tela, listas reais (`<ul>/<li>`) para alunos e histórico.
- **Skip-link:** link "Pular para o conteúdo", oculto até receber foco por teclado.
- **Notificações:** o toast usa `role="status"` e `aria-live="polite"` — é anunciado sem roubar o foco do usuário.
- **Contraste de cor:** texto secundário $\geq 6,2:1$, badges de risco $\geq 4,9:1$, texto do botão primário $5,1:1$ — todos medidos sobre seus fundos reais.
- **Movimento reduzido:** a entrada animada do toast respeita `prefers-reduced-motion`.
- **Autopreenchimento:** campo de e-mail com `type="email"`; senha com `autocomplete="current-password"` para gerenciadores de senha.

9. Como Executar

9.1 Versão .html

Nenhuma instalação necessária. Duas opções:

```
# opção 1 – abrir direto
open escolaalerta-prototipo.html

# opção 2 – servidor estático local
# (recomendado para evitar restrições de file:// em alguns navegadores)
python3 -m http.server 8000
# depois abrir http://localhost:8000/escolaalerta-prototipo.html
```

9.2 Versão .jsx

Requer um projeto React já configurado (Vite, Create React App ou Next.js) com a dependência `lucide-react` instalada:

```
npm install lucide-react

// main.jsx
import App from "./escolaalerta-prototipo.jsx";
// renderizar <App /> na raiz do projeto
```

10. Limitações Conhecidas

Registradas deliberadamente para deixar claro o que este protótipo demonstra e o que fica para a integração com o back-end descrito nos demais documentos do trabalho.

- **Sem persistência:** todo estado é `useState` em memória; recarregar a página apaga qualquer cadastro, registro ou a preferência de tema claro/escuro feitos na sessão.
- **Sem autenticação:** o login apenas escolhe qual perfil simular — não há verificação de credenciais nem sessão real.
- **Responsável fixo:** a jornada do responsável está presa a `ALUNO_DEMO_ID` ("a1"); não há seleção entre múltiplos dependentes.
- **Plano de ação não retroativo:** alunos já em risco nos dados semeados só recebem plano de ação após o próximo indicador ser registrado.
- **Duas fontes divergentes:** `.html` e `.jsx` são mantidos manualmente em paralelo — uma alteração visual precisa ser replicada nos dois arquivos.
- **Sem testes automatizados** no front-end: a estratégia de testes do projeto (unitário/integração/sistema) está descrita em "Plano e Execução de Testes de Software".

11. Capturas de Tela

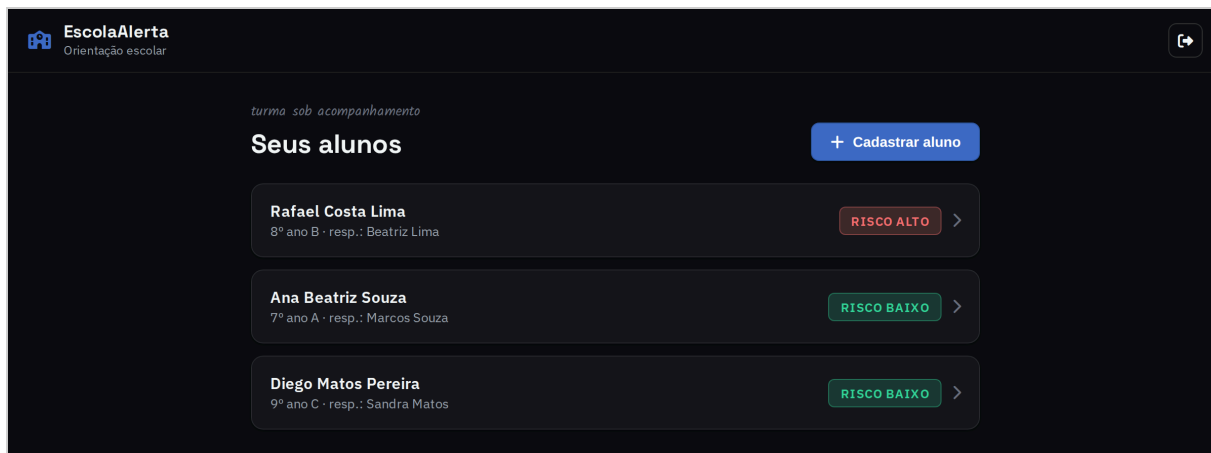


Figura 3: Painel do orientador — lista de alunos sob acompanhamento, com badge de risco por aluno.

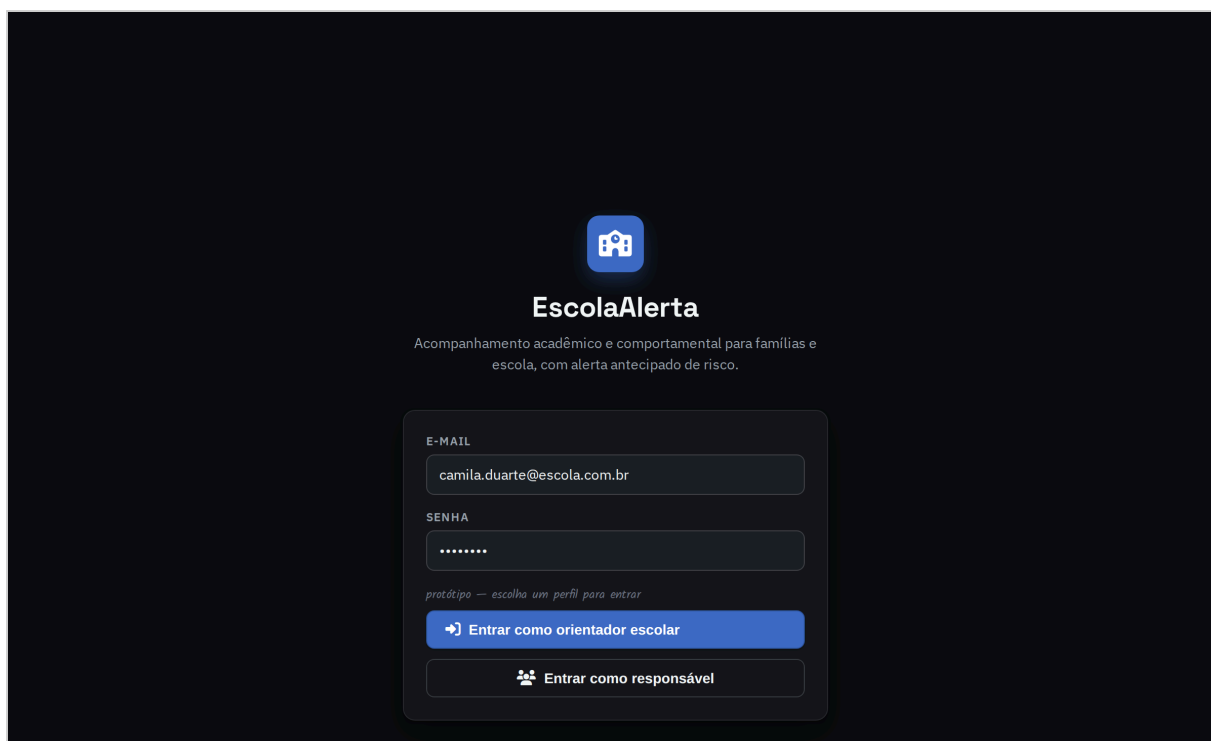


Figura 4: Tela de login (seleção de perfil de simulação).

←

EscolaAlerta
Orientação escolar

↔

registro de indicador

Rafael Costa Lima

8º ano B RISCO ALTO

Notas

Frequência

Ocorrências

Saídas

DISCIPLINA

ex.: Matemática

NOTA (0-10)

PERÍODO

3º bimestre

Registrar nota

Preencha disciplina e nota para habilitar o registro.

Histórico registrado

Matemática: nota 5.2 (3º bimestre)

Português: nota 5.8 (3º bimestre)

Falta em 12/08

Falta em 14/08

Falta em 19/08

Não entregou trabalho em grupo - gravidade leve

Figura 5: Registro de indicador para um aluno, com abas por tipo e histórico registrado.